Кодировки текста: запись и чтение файла в UTF-8

1. Назначение работы

В этой работе рассматривается, как Python записывает текст в файл и читает его обратно с указанной кодировкой. Основное внимание уделяется UTF-8: текст записывается, читается, сравнивается с исходной строкой и дополнительно рассматривается его байтовое представление.
Работа рассчитана на 2 академических часа. Все действия выполняются в Python 3.x в среде Visual Studio Community только внутри собственной рабочей папки проекта.

> Важно
> Все примеры выполняются только с тестовыми текстовыми файлами внутри собственной рабочей папки проекта. Не изменяйте системные настройки кодировок Windows и не используйте системные или чужие пользовательские файлы.

2. Используемое программное обеспечение

Компонент	Назначение
Python 3.x	выполнение программы и работа со стандартной библиотекой
Visual Studio Community	создание, запуск и проверка Python-проекта
pathlib.Path	формирование безопасного пути к рабочей папке и текстовым файлам
open(), encode(), decode()	запись и чтение текста, преобразование строки в байты и обратно

3. Порядок выполнения работы

3.1. Подготовка безопасной рабочей папки

Сначала подготовьте отдельную тестовую папку. Она нужна, чтобы все файлы с разными вариантами текста создавались только внутри проекта и не затрагивали пользовательские документы.

1. Создайте или откройте Python-проект в Visual Studio Community.
2. Убедитесь, что файл main.py запускается без ошибок.
3. Выведите Path.cwd(), чтобы увидеть текущую рабочую папку.
4. Все текстовые файлы этой работы создавайте только внутри Path.cwd().

Подготовка учебной папки

```
from pathlib import Path

base = Path.cwd() / "encoding_demo"
texts = base / "texts"
results = base / "results"
file_path = texts / "greeting.txt"

texts.mkdir(parents=True, exist_ok=True)
results.mkdir(parents=True, exist_ok=True)
text = "Привет, мир!\nФайл сохранён в UTF-8"

with open(file_path, "w", encoding="utf-8") as file:
    file.write(text)
print("Файл создан:", file_path)

print("Кодировка при записи: UTF-8")
```

После запуска внутри encoding_demo появятся папки texts и results. В texts будет создан greeting.txt с кириллическим текстом, записанным с явным указанием encoding="utf-8".

Ожидаемая структура после запуска

```
encoding_demo
├── texts
│   └── greeting.txt
└── results
```

Что показывает этот пример:
Path.cwd() задаёт безопасную рабочую область. Папка encoding_demo создаётся внутри проекта, а greeting.txt записывается в UTF-8. Параметр encoding задаёт правило преобразования символов строки в байты файла.

Задание 1. Подготовить собственный UTF-8 файл

1. Создайте внутри Path.cwd() каталог encoding_lab.
2. Внутри encoding_lab создайте папку texts.
3. Программно создайте texts/student.txt и запишите в него минимум две строки с кириллицей, используя encoding="utf-8".
4. Откройте созданный файл в Проводнике или текстовом редакторе и убедитесь, что кириллица отображается правильно.

3.2. Запись текста в UTF-8

При записи текста Python преобразует символы строки в последовательность байтов. Параметр encoding="utf-8" явно указывает, по какому правилу выполнять это преобразование.

Минимальный синтаксис

```
with open(file_path, "w", encoding="utf-8") as file:

    file.write(text)
```

Пример: запись заметки в UTF-8

```
from pathlib import Path
base = Path.cwd() / "encoding_demo"

file_path = base / "texts" / "note.txt"
file_path.parent.mkdir(parents=True, exist_ok=True)
text = "Тема: кодировки\nСтатус: файл записан"

with open(file_path, "w", encoding="utf-8") as file:

    file.write(text)
print("Файл существует:", file_path.exists())
```

Разбор результата:
После записи note.txt появляется в папке texts. Кириллица сохраняется корректно, потому что при преобразовании строки в байты явно использована UTF-8.

Задание 2. Записать собственный текст в UTF-8

1. Сформируйте путь encoding_lab/texts/profile.txt.
2. Подготовьте строку минимум из трёх строк: имя, группа и короткая фраза на русском языке.
3. Запишите текст через open(..., "w", encoding="utf-8").
4. Проверьте через exists(), что файл создан.
5. Откройте файл и зафиксируйте корректное отображение кириллицы.

3.3. Чтение текста в UTF-8

При чтении выполняется обратное преобразование: байты файла превращаются в символы строки. Для корректного результата указывается та же кодировка, которая использовалась при записи.

Общая форма

```
with open(file_path, "r", encoding="utf-8") as file:
    result = file.read()
```

Пример: чтение note.txt

```
from pathlib import Path

base = Path.cwd() / "encoding_demo"
file_path = base / "texts" / "note.txt"
expected = "Тема: кодировки\nСтатус: файл записан"

with open(file_path, "r", encoding="utf-8") as file:

    result = file.read()
print(result)\nprint("Текст совпадает:", result == expected)
```

Как проверить результат:
Если при записи и чтении указана UTF-8, прочитанная строка должна совпасть с исходной. Сравнение result == expected даёт True и подтверждает корректное преобразование.

Задание 3. Прочитать profile.txt

1. Сформируйте путь encoding_lab/texts/profile.txt.
2. Прочитайте файл через open(..., "r", encoding="utf-8").
3. Выведите прочитанный текст в консоль.
4. Сравните прочитанный текст с исходной строкой.
5. В отчёте зафиксируйте результат сравнения и объясните, почему одинаковая кодировка дала корректный текст.

3.4. Как строка превращается в байты

Кодировка задаёт соответствие между символами и байтами. Метод encode() превращает строку Python в объект bytes, а decode() выполняет обратное преобразование.

Минимальный синтаксис

```
# encode() и decode() являются методами стандартных типов

data = text.encode("utf-8")
```

Пример: байтовое представление строки

```
from pathlib import Path
# encode() и decode() являются методами стандартных типов

base = Path.cwd() / "encoding_demo"
text = "Системное программирование"
data = text.encode("utf-8")

restored = data.decode("utf-8")

print("Байты:", data)
print("Восстановленный текст:", restored)
```

Разбор результата:
Байты могут выглядеть непривычно, но это представление той же строки. Если декодировать их той же UTF-8, получится исходный текст без изменений.

Задание 4. Получить байты другой строки

1. Создайте строку «Кодировка UTF-8 работает с кириллицей».
2. Преобразуйте строку в bytes через encode("utf-8").
3. Выведите объект bytes в консоль.
4. Декодируйте bytes обратно через decode("utf-8").
5. Сравните восстановленную строку с исходной и зафиксируйте результат.

3.5. Почему кодировка должна совпадать

Одни и те же байты можно попытаться интерпретировать по разным правилам. Если байты UTF-8 декодировать другой кодировкой, символы могут исказиться, хотя сами байты при этом не изменяются.

Сравнение правильного и неправильного декодирования

```
text = "Привет"
data = text.encode("utf-8")
correct = data.decode("utf-8")
wrong = data.decode("latin-1")
print("UTF-8:", correct)
print("latin-1:", wrong)
print("Совпадает:", correct == text)
print("Байты:", data)
```

Почему результат отличается:
correct восстанавливает исходное слово, потому что для encode() и decode() использована UTF-8. latin-1 трактует те же байты по другим правилам, поэтому получается искажённая строка.

Задание 5. Сравнить две кодировки

1. Создайте строку «Файл и кодировка».
2. Получите её байты через encode("utf-8").
3. Декодируйте одни и те же байты через utf-8 и latin-1.
4. Зафиксируйте оба результата и объясните различие одной-двумя фразами.

3.6. Чтение файла как набора байтов

Иногда полезно увидеть содержимое текстового файла до декодирования. Метод Path.read_bytes() возвращает объект bytes без автоматического преобразования в строку.

Пример чтения байтов файла

```
from pathlib import Path
file_path = Path.cwd() / "encoding_demo" / "texts" / "note.txt"
data = file_path.read_bytes()
print("Первые байты:", data[:20])
text = data.decode("utf-8")
print("Текст:")
print(text)
print("Размер в байтах:", len(data))
```

> Что демонстрирует пример:
> read_bytes() возвращает байты без декодирования. Чтобы получить обычную строку, байты явно преобразуются через decode("utf-8"). len(data) показывает размер содержимого в байтах.

Задание 6. Прочитать собственный файл как bytes

1. Прочитайте encoding_lab/texts/profile.txt через read_bytes().
2. Выведите первые 20 байтов файла.
3. Декодируйте полученные байты через utf-8 и выведите строку.
4. Сравните декодированный текст с результатом обычного чтения файла.

4. Полный учебный пример

Ниже запись, чтение, получение байтов и два варианта декодирования собраны в одной программе. Учебный пример использует папку encoding_demo, а самостоятельная работа — другие имена и другой текст.

Полный пример «Проверка UTF-8»

```
from pathlib import Path
# Используется только стандартная библиотека Python

base = Path.cwd() / "encoding_demo"
texts = base / "texts"
```

```
texts.mkdir(parents=True, exist_ok=True)
file_path = texts / "lesson.txt"

# Файл создаётся внутри подготовленной папки
original_text = "Кодировка UTF-8\nТекст читается корректно"
with open(file_path, "w", encoding="utf-8") as file:

    file.write(original_text)
with open(file_path, "r", encoding="utf-8") as file:
    loaded = file.read()

data = file_path.read_bytes()
decoded_utf8 = data.decode("utf-8")
decoded_latin1 = data.decode("latin-1")

print("Исходный текст:", original_text)
print("Прочитанный текст:", loaded)
print("Совпадает:", loaded == original_text)

print("Размер в байтах:", len(data))
print("UTF-8:", decoded_utf8)
print("latin-1:", decoded_latin1)

print("UTF-8 совпадает:", decoded_utf8 == original_text)
print("Первые байты:", data[:20])
print("Файл:", file_path)
```

Разбор учебного примера:
1) Текст записывается в файл с encoding="utf-8". 2) Чтение с той же кодировкой восстанавливает исходную строку. 3) read_bytes() показывает байтовое содержимое файла. 4) UTF-8 возвращает исходный текст, а latin-1 интерпретирует те же байты иначе.

5. Итоговая самостоятельная работа

Создайте программу «Проверка кодировки текстового файла». Она должна записать собственный текст в UTF-8, прочитать его обратно, проверить совпадение, получить байты и сравнить правильное и неправильное декодирование.

Структура самостоятельной работы

```
encoding_check
└── note.txt
```

В encoding_check программно создайте note.txt. В файле должно быть минимум три строки текста, включая кириллицу.

Что выполнить

1. Создайте строку минимум из трёх строк и запишите её в note.txt с encoding="utf-8".
2. Прочитайте note.txt с encoding="utf-8" и выведите содержимое.
3. Сравните прочитанный текст с исходной строкой.
4. Получите байты файла через read_bytes() и выведите первые 20 байтов.
5. Декодируйте байты через utf-8 и проверьте совпадение с исходным текстом.
6. Декодируйте те же байты через latin-1 и выведите результат.
7. Кратко объясните, почему результаты utf-8 и latin-1 отличаются.
8. Сделайте скриншоты файла и консоли с результатами проверки.

> Важно
> Текст самостоятельной работы должен отличаться от учебного примера. Используйте ту же последовательность действий, но выберите собственное содержимое note.txt и не копируйте строки примера дословно.

6. Самостоятельное изменение

Создайте дополнительный файл extra.txt с другим русским текстом. Запишите его в UTF-8, получите bytes и сравните количество символов в строке с количеством байтов.

В отчёте зафиксируйте текст, количество символов, количество байтов и кратко объясните, почему эти значения могут отличаться.

7. Требования к отчёту

Отчёт оформляется в Microsoft Word. В нём должны быть видны фрагменты кода, текстовый файл, результаты чтения и сравнение вариантов декодирования.

Раздел отчёта	Что приложить
Тема и цель	название работы и краткая цель
UTF-8 файл	скриншот note.txt и его содержимого
Запись UTF-8	фрагмент кода + подтверждение результата
Чтение UTF-8	фрагмент кода + подтверждение результата
Байты и декодирование	фрагмент кода + подтверждение результата
Итоговая программа	полный код текстом с сохранением отступов
Самостоятельное изменение	код, текст, количество символов и байтов
Вывод	3–5 предложений о UTF-8, байтах и совпадении кодировок

> Важно для отчёта
> Код итоговой программы вставляется в Word текстом с сохранением отступов. Каждый скриншот подпишите «Рисунок N — ...» и под ним кратко поясните, какая кодировка использована и что подтверждает результат.

7.1. Таблица наблюдений

Заполните таблицу только по фактическим результатам своей программы.

Наблюдение	Фактическое значение	Результат
Путь к note.txt		
Исходный текст		
Прочитанный текст		
Совпадение строк		
Количество байтов		
Результат latin-1		
Самостоятельное изменение		

8. Типичные ошибки

• файл создаётся не внутри рабочей папки проекта;

• при записи не указана encoding="utf-8";
• при чтении указана другая кодировка и текст искажается;
• результат чтения не сравнивается с исходной строкой;
• bytes принимаются за отдельный текст, а не за байтовое представление строки;
• encode() и decode() используются с разными кодировками без анализа результата;
• в отчёте показан только код, но нет фактического результата чтения и декодирования.

9. Контрольные вопросы

1. Что такое кодировка текста?
2. Для чего в open() указывается encoding="utf-8"?
3. Что делает метод encode()?
4. Что делает метод decode()?
5. Почему кодировка при записи и чтении должна совпадать?
6. Чем строка str отличается от объекта bytes в этой работе?
7. Что возвращает Path.read_bytes()?
8. Почему декодирование UTF-8-байтов через latin-1 даёт другой текст?

10. Критерии готовности

Проверка	Да / нет
Рабочая папка создана только внутри Path.cwd()	
Текстовый файл записан с encoding="utf-8"	
Файл прочитан с encoding="utf-8"	
Прочитанный текст сравнен с исходной строкой	
Получено байтовое представление текста	
Показаны результаты декодирования utf-8 и latin-1	
Выполнено самостоятельное изменение	
В отчёте есть код, результаты, ответы и вывод	